Name:Abhishek Yadav.USN No.:CS23071 Aim:To implement the Random Forest algorithm and evaluate the model performance using appropriate metrics.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
import pandas as pd
path="/content/drive/MyDrive/ml_dataset/banking.csv"
df=pd.read_csv(path)
df.head(5)
```

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previ
0	44	blue-collar	married	basic.4y	unknown	yes	no	cellular	aug	thu	...	1	999	
1	53	technician	married	unknown	no	no	no	cellular	nov	fri	...	1	999	
2	28	management	single	university.degree	no	yes	no	cellular	jun	thu	...	3	6	
3	39	services	married	high.school	no	no	no	cellular	apr	fri	...	2	999	
4	55	retired	married	basic.4y	no	yes	no	cellular	aug	fri	...	1	3	

5 rows × 21 columns

```
df.shape
```

```
(41188, 21)
```

```
df.columns
```

```
Index(['age', 'job', 'marital', 'education', 'default', 'housing', 'loan',
       'contact', 'month', 'day_of_week', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'emp_var_rate', 'cons_price_idx',
       'cons_conf_idx', 'euribor3m', 'nr_employed', 'y'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df.isnull().sum()
```

	0
age	0
job	0
marital	0
education	0
default	0
housing	0
loan	0
contact	0
month	0
day_of_week	0
duration	0
campaign	0
pdays	0
previous	0
poutcome	0
emp_var_rate	0
cons_price_idx	0
cons_conf_idx	0
euribor3m	0
nr_employed	0
y	0

dtype: int64

```
df[df.isnull().any(axis = 1)]
```

age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous	poutcome

0 rows × 21 columns

```
x = df.drop(columns=['y'])
y = df['y']
```

```
x.head()
```

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	duration	campaign	pdays
0	44	blue-collar	married	basic.4y	unknown	yes	no	cellular	aug	thu	210	1	999
1	53	technician	married	unknown	no	no	no	cellular	nov	fri	138	1	999
2	28	management	single	university.degree	no	yes	no	cellular	jun	thu	339	3	6
3	39	services	married	high.school	no	no	no	cellular	apr	fri	185	2	999
4	55	retired	married	basic.4y	no	yes	no	cellular	aug	fri	137	1	3

```
x = pd.get_dummies(x, dtype = int)
```

```
x.head()
```

	age	duration	campaign	pdays	previous	emp_var_rate	cons_price_idx	cons_conf_idx	euribor3m	nr_employed	...	month_
0	44	210	1	999	0	1.4	93.444	-36.1	4.963	5228.1	...	
1	53	138	1	999	0	-0.1	93.200	-42.0	4.021	5195.8	...	
2	28	339	3	6	2	-1.7	94.055	-39.8	0.729	4991.6	...	
3	39	185	2	999	0	-1.8	93.075	-47.1	1.405	5099.1	...	
4	55	137	1	3	1	-2.9	92.201	-31.4	0.869	5076.2	...	

5 rows × 63 columns

```
x.head()
```

	age	duration	campaign	pdays	previous	emp_var_rate	cons_price_idx	cons_conf_idx	euribor3m	nr_employed	...	month_
0	44	210	1	999	0	1.4	93.444	-36.1	4.963	5228.1	...	
1	53	138	1	999	0	-0.1	93.200	-42.0	4.021	5195.8	...	
2	28	339	3	6	2	-1.7	94.055	-39.8	0.729	4991.6	...	
3	39	185	2	999	0	-1.8	93.075	-47.1	1.405	5099.1	...	
4	55	137	1	3	1	-2.9	92.201	-31.4	0.869	5076.2	...	

5 rows × 63 columns

```
x.shape
```

```
(41188, 63)
```

```
x.columns
```

```
Index(['age', 'duration', 'campaign', 'pdays', 'previous', 'emp_var_rate',
       'cons_price_idx', 'cons_conf_idx', 'euribor3m', 'nr_employed',
       'job_admin.', 'job_blue-collar', 'job_entrepreneur', 'job_housemaid',
       'job_management', 'job_retired', 'job_self-employed', 'job_services',
       'job_student', 'job_technician', 'job_unemployed', 'job_unknown',
       'marital_divorced', 'marital_married', 'marital_single',
       'marital_unknown', 'education_basic.4y', 'education_basic.6y',
       'education_basic.9y', 'education_high.school', 'education_illiterate',
       'education_professional.course', 'education_university.degree',
       'education_unknown', 'default_no', 'default_unknown', 'default_yes',
       'housing_no', 'housing_unknown', 'housing_yes', 'loan_no',
       'loan_unknown', 'loan_yes', 'contact_cellular', 'contact_telephone',
       'month_apr', 'month_aug', 'month_dec', 'month_jul', 'month_jun',
       'month_mar', 'month_may', 'month_nov', 'month_oct', 'month_sep',
       'day_of_week_fri', 'day_of_week_mon', 'day_of_week_thu',
       'day_of_week_tue', 'day_of_week_wed', 'poutcome_failure',
       'poutcome_nonexistent', 'poutcome_success'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y, train_size = 0.7, test_size=0.3, random_state = 42)
x_train.shape, x_test.shape
```

```
((28831, 63), (12357, 63))
```

```
from sklearn.ensemble import RandomForestClassifier
rf_classifier = RandomForestClassifier(n_estimators=100, n_jobs=-1,max_depth=5, oob_score=True, random_state=42)
rf_classifier.fit(x_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=5, n_jobs=-1, oob_score=True, random_state=42)
```

```
y_pred = rf_classifier.predict(x_test)
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
print('confusion matrix')
cm=confusion_matrix(y_test, y_pred)
print(confusion_matrix(y_test, y_pred))
```

```
confusion matrix
[[10848    83]
```

```
print('Accuracy Score:')
print(accuracy_score(y_test, y_pred))
```

```
Accuracy Score:
0.8995710933074371
```

```
print(rf_classifier.oob_score)
```

```
True
```

```
print('Classification Report:')
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.99      0.95     10931
           1       0.76      0.19      0.30      1426

    accuracy                           0.90     12357
   macro avg       0.83      0.59      0.62     12357
weighted avg       0.89      0.90      0.87     12357
```

```
rf = RandomForestClassifier(random_state=42, n_jobs=-1)

params = {
    'max_depth': [2,3,5,10,20],
    'min_samples_leaf': [5,10,20,50,100,200],
    'n_estimators': [10,25,30,50,100,200]
}

from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(estimator=rf, param_grid=params, cv=4, n_jobs=-1, verbose=1, scoring='accuracy')

grid_search.fit(x_train, y_train)
```

```
Fitting 4 folds for each of 180 candidates, totalling 720 fits
```

▸ **GridSearchCV**